BirdCLEF-2026 — Weekly Progress

Anthony Miyaguchi

2026-02-01

Week of 2026-01-26

- Ported SSAMBA to HuggingFace-compatible model (experiment 004)
- Fixed critical bidirectional loop bug (18.4% $\rightarrow$ 61.6% ESC-50)
- Matched original training recipe — LoRA & full finetune hit 87.0% fold 1
  - Exceeds published 85.5% (5-fold mean)
- Profiled SSAMBA-HF inference: CUDA kernels 22–130$\times$ faster than PyTorch fallback
- Built ONNX custom op for Mamba selective scan — ~100$\times$ CPU speedup

SSAMBA-HF Port (Experiment 004)

- Custom `BidirectionalMamba` wrapped in HuggingFace `PreTrainedModel` API
- Numerical equivalence verified:
  - Cosine similarity = 1.000000 with original
  - Max absolute diff = 5.25e-05
- Eliminates CUDA version / timm dependency issues
- Enables standard `from_pretrained()` / `save_pretrained()` workflow

Bidirectional Bug Fix

- Our initial port was missing the outer bidirectional loop
  - Even layers: forward pass
  - Odd layers: reversed sequence
- Verified against reference implementation
- Impact: **18.4% → 61.6%** linear probe accuracy on ESC-50

ESC-50 Benchmark

Encoder Benchmark (Linear Probe)

Encoder	Accuracy
BEATs	95.8%
AST	95.5%
EAT	77.9%
SSAMBA-HF-small	61.6%
wav2vec2	48.2%

All frozen features, linear probe on ESC-50.

SSAMBA Finetuning

Method	Fold 1 Accuracy
Linear probe	61.6%
LoRA r=8	87.0%
Full finetune	87.0%

- Exceeds published SSAMBA-small 85.5% (5-fold mean)
- 9 recipe differences fixed
- Full 5-fold CV still pending

Inference Profiling

Configuration	Latency (batch=1)	Throughput
SSAMBA CUDA kernels	30 ms	110 samples/sec
SSAMBA PyTorch FB	3,908 ms	0.3 samples/sec
BEATs GPU	8 ms	367 samples/sec

- CUDA kernels **22–130×** faster than PyTorch fallback
- SSAMBA ~3–4× slower than BEATs but ~3× less GPU memory
- Mamba layers = 93.5% of inference time

mamba-onnx: Custom Op for Selective Scan

- Built ONNX custom operator for Mamba selective scan (~100× CPU speedup)
- Zig + C++ backends, both built by default
- OpenMP (C++) and ForkJoin parallel dispatch (Zig)
- Docker cross-compilation + GitHub Actions release workflow
- pytest-benchmark for statistically rigorous comparisons

mamba-onnx: Backend Speedup

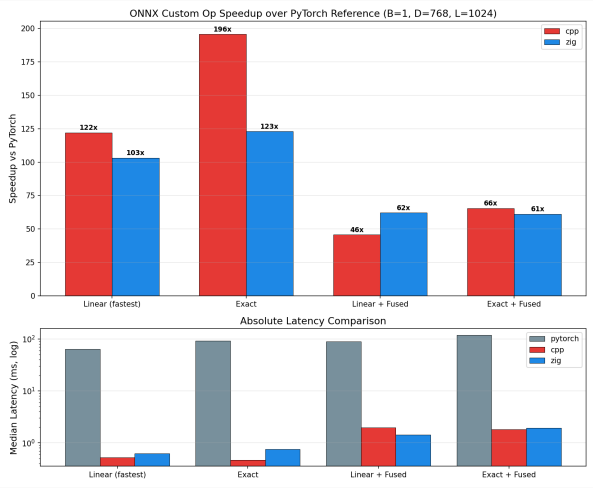


Figure 1: Speedup

mamba-onnx: Sequence Length Scaling

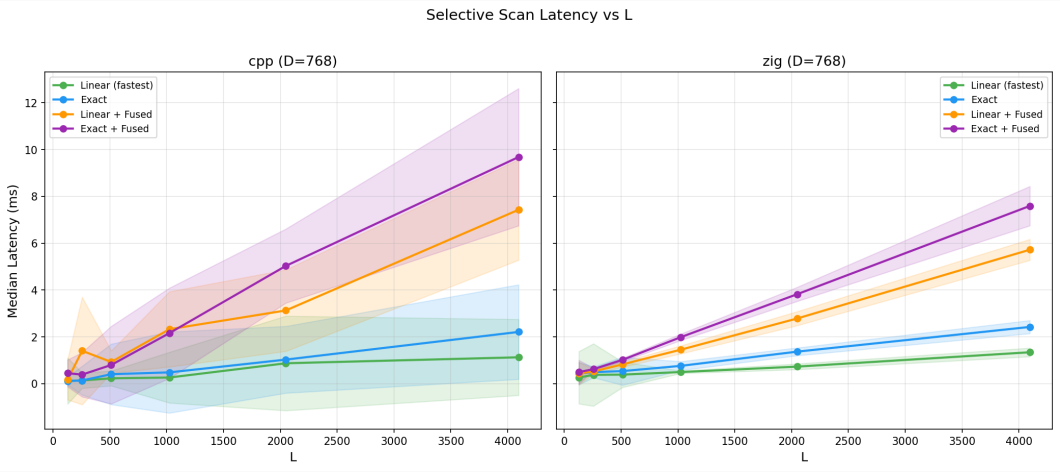


Figure 2: Scaling L

Concept Docs

Token Architecture

- Discrete audio tokens (WavTokenizer: 40–75 tokens/sec) instead of spectrogram patches
- 60s audio = ~4,500 tokens — tractable for Mamba, expensive for attention
- Bidirectional Mamba for full-sequence context
- Multi-task: classification + SED + reconstruction heads

Pretraining Compute Feasibility

- PACE budget: ~200–500 GPU-hours
- Strategy ladder: distillation (10–50h) → patch surgery (5–20h) → small SSL (50–200h)

Next Steps

- Benchmark SSAMBA on CPU with custom op — different token configurations
- Determine viable sequence length for Kaggle kernel constraints